

제목: 신호등 인식 기반 TurtleBot 자율주행
구현 및 디버깅 보고서.

작성자:20211115 정재준

제출일: 2025-12-08

프로젝트 목표.

1. 터틀봇 구동 노드 작성.

(A) 터틀봇이 움직이는 대략적인 환경은 다음과 같다.

(B) 터틀봇은 **S** 지점에서 첫 번째 신호등 **1** 방향으로 출발한다. (이 방향으로 터틀봇을 놓고 시작한다)

(C) 신호등 **1** 앞에서 일단 멈춘다. (신호등이 놓인 물체 앞 **30cm** 전방)

(D) 빨간색 신호등을 인식하면 왼쪽으로 **90°**만큼 회전 후 직진하고, 녹색 신호등을 인식하면 오른쪽으로 **90°**만큼 회전 후 직진한다.

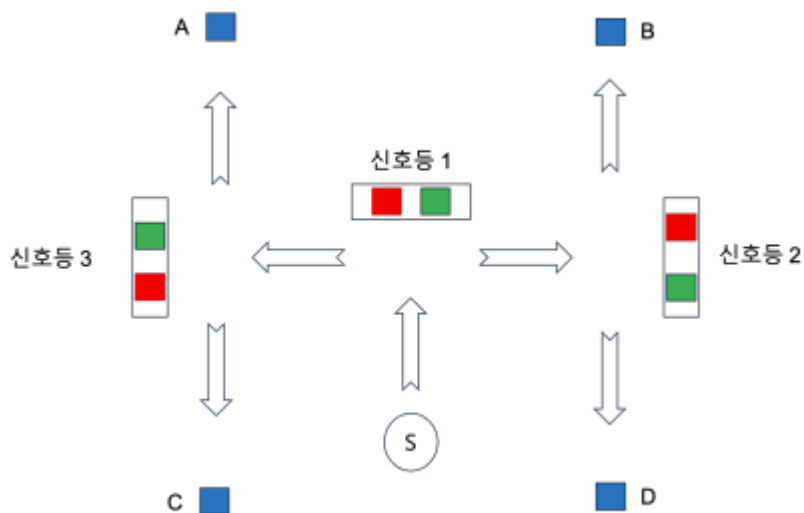
(E) 신호등 **2** 또는 신호등 **3** 방향으로 진행하여 신호등이 놓인 물체 앞 **30cm** 전방에서 일단 멈춘다.

(F) 빨간색 신호등을 인식하면 왼쪽으로 **90°**만큼 회전 후 직진하고, 녹색 신호등을 인식하면 오른쪽으로 **90°**만큼 회전 후 직진한다.

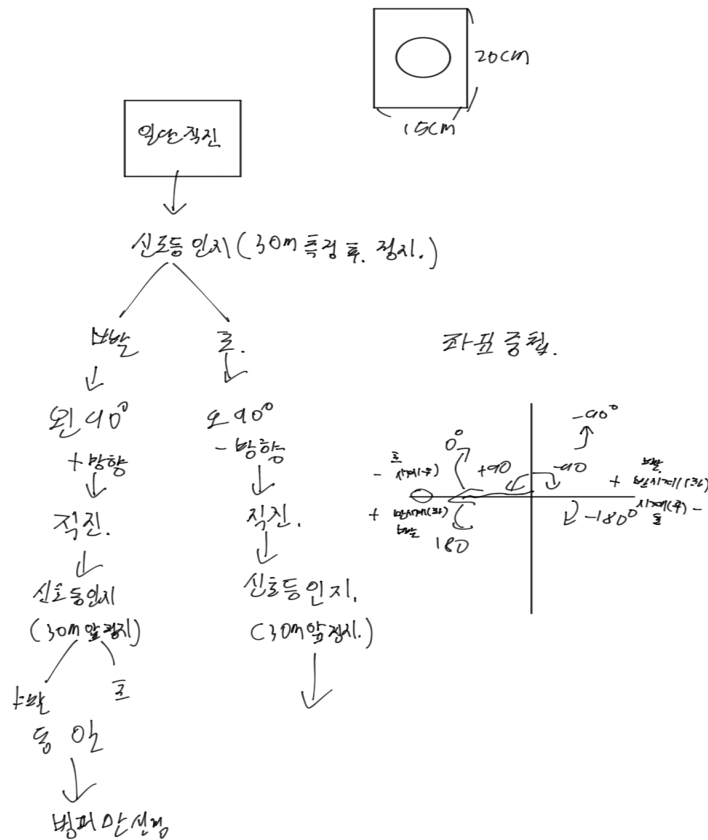
(G) **A** 또는 **B** 또는 **C** 또는 **D** 방향으로 직진한다.

(H) 파란색 신호가 놓인 물체에 부딪히면 정지한다.

(I) 신호등은 **A4** 용지 위에 프린트된 가로 **15cm**, 세로 **20cm** 의 사각형 형태이다. (시연 할때마다 신호등 배치가 매번 다름)



터틀봇 제어 코드 짜기 전 간단한 구상



이전에도 혼자서 프로젝트를 진행할 때 이렇게 직접 손으로 구상을 정리 한 뒤 코드를 작성하면 수월하다는 것을 경험한 바탕이 있어 위 사진처럼 알고리즘을 작성한 뒤 코드를 작성함.

우선 코드를 실행하기 전 터틀봇이 작동하기 위해서는 영상처리 코드부터 잘 작동해야 제어 코드가 잘 작동한다는 걸 알고 있었기 때문에, 디버깅을 위해 조찬형 학생이 준 코드를 기반으로 주석 처리 및 알고리즘 해석..

영상처리 코드를 해석한 뒤 이를 토대로 카메라 입력만을 독립적으로 확인할 수 있는 전용 디버깅 코드를 작성함.

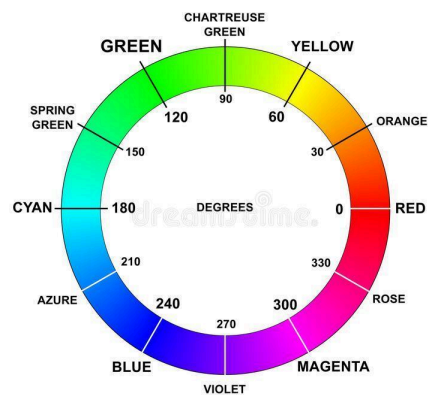
카메라 관련 코드 문제 1.

코드를 터틀봇에 업로드하고 카메라 실행 노드를 켜서 어떻게 인식되는지 확인할 결과 초록색과 빨간색을 잘 구분을 못하는 걸 확인(초록색인데 빨간색으로 인식을 많이 함).

카메라 거리도 45cm 밑으로 내려가면 거리 인식을 못 하는 상황이 나옴.

카메라 관련 코드 디버깅 1.

밑에 그림처럼 HSV가 이루어져 있기 때문에 RED 마스크를 두 구간으로 나누어 한 건 문제가 없다고 생각.



우리가 사용하는 초록색 그림은 밝은 초록색이 아니라 어둡다는 걸 인지.

초록색의 범위를 보니 너무 밝은 초록색이 포함되어 있고 채도와 명도도 밝다고 판단 됨.

디버깅은 초록색의 색상 채도 명도를 조금씩 낮추면서 터미널 로그를 통해 적당한 값을 찾음.

카메라 거리 문제는 먼저 카메라 스펙을 확인해 보니 대략 43cm 이하의 거리는 측정을 못 하는 걸 확인할 수 있었음.

이후 교수님께 상황을 설명해 드린 결과 물체를 인식하고 멈춰야 하는 30cm 이내에서 65cm~80cm 범위로 수정해 주셔서 이에 대한 문제도 해결함.

제어 코드 관련 문제 1.

이제 터틀봇에 코드를 올려 실제로 작동을 시켜보니 직진을 못 하고 약간 우측으로 치우친 상태로 직진하는 걸 확인.

제어 코드 관련 코드 디버깅 1.

프로젝트 2번을 진행하면서 사용했던 피드백 코드를 직진 성분을 하기 위한 코드로 바꿔주고 넣어주면 된다고 생각함.

목표 각도를 시작점 터틀봇의 각도로 지정하고 현재 터틀봇의 각도와 시작점 거북이 봇의 각도 차이를 p 값을 곱하고 yaw 값에 더해서 보정함.

디버깅을 하던 중 터틀봇의 목표 각도를 합쳐서 조건, 위치마다 각도를 설정해 제어를 하는 것 보다, 회전 이후 각도를 측정해 그 각도에 $+90$ 도를 해주는 방법이 더 깔끔하겠다는 생각을 해 코드를 수정하기 시작함.

추가한 주요 함수 및 코드.

디버깅 1.1 색 판단 이후 회전을 위한 코드.

```
void rotate_angle(ros::Publisher& pub, double deg, ros::Rate& looprate)
{
    double target = setrad(turtle_theta + deg * M_PI/180.0); //회전할 각도를
    라디안 변환.
    double w = 1; //각속도의 절댓값.
    geometry_msgs::Twist cmd;
    cmd.linear.x = 0.0; //회전 중에는 선속도는 0이다.

    while(ros::ok()){
        ros::spinOnce();
        double diff = setrad(target - turtle_theta); //목표값과 현재 터틀봇의
        각도를 뺀.
        if(fabs(diff) < 0.05) break; //그 값의 절댓값이 0.05보다 작으면
        while문은 끝.
        cmd.angular.z = (diff > 0 ? w : -w); //차이에 따라서 각속도 해야해서
        차이가 0보다 크면 w 아니면 -w
        pub.publish(cmd);
        looprate.sleep();
    }
    // 정지
    cmd.angular.z = 0.0;
    pub.publish(cmd);
}
```

디버깅 1.2 정확한 직진을 위해 추가 및 변경한 코드.

```
const double P_yaw = 1.5; //P값 튜닝 결국 1.5로 결정.
```

```
double target_yaw = 0.0; //직진을 위한 타겟 yaw값은 0
```

```
double yaw_e = setrad(target_yaw - turtle_theta);  
msg.angular.z = P_yaw * yaw_e;
```

제어 코드 관련 문제 2.

코드를 수정하고 실행을 해보니 첫 회전에서 잘 작동하는데, 회전한 이후 터틀봇이 이상하게 진행하는 걸 확인.

제어 코드 관련 코드 디버깅 2.

코드를 확인 한 결과 회전을 한 뒤 터틀봇의 목표 각도를 회전한 뒤에 각도로 다시 업데이트해 줘야 한다는 걸 알게 됨.

디버깅 2. 추가한 코드.

```
target_yaw = turtle_theta; //타겟 yaw값을 현재 터틀봇에 맞춤(회전했으니까 직진을  
위해 목표 업데이트)
```

제어 코드 관련 문제 3.

이후 잘 작동은 하지만 마지막 장애물에 부딪혀야 하는데, 장애물 거리 인식 조건문 때문에 물체 앞에서 멈추는 문제 발생.

제어 코드 관련 코드 디버깅 3.

거리 인식하는 영상처리 코드에 두 번째 신호등 처리 완료 플래그가 false일 때만 작동하도록 조건문을 넣음.

디버깅 3. 추가한 코드

```
if(second_traffic_light == false){ //2번째 신호등 처리를 한 뒤에는 거리 감지  
off  
    traffic_light_30cm = (distance_cm > 65 && distance_cm < 80); //거리가  
65cm~80cm면 장애물 감지 플래그 on  
}  
else{  
    traffic_light_30cm = false; //아니면 false  
}
```

제어 코드 관련 문제 4.

신호등을 처리하는 중에 사람에 의해 범퍼가 부딪치면 미션을 완수하기 전 코드가 정지되는 걸 확인.

제어 코드 관련 코드 디버깅 4.

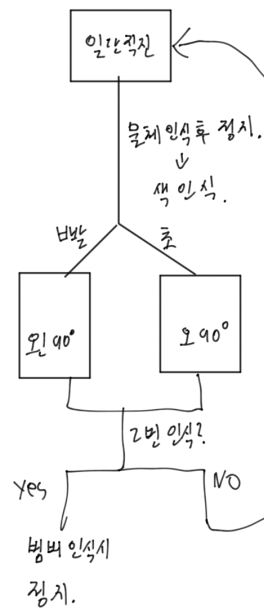
첫 번째 신호등과 두 번째 신호등 제어 코드를 완료한 상태에서만 범퍼가 부딪치면 코드가 종료되게 조건문을 추가.

디버깅 4. 조건문 추가.

```
if(first_traffic_light == true && second_traffic_light == true &&
bumper_state == true){ //1,2 신호등 처리 완료했고 범퍼 상태가 부딪치면 (정지명령
실행.)
    msg.linear.x = 0.0;
    msg.angular.z = 0.0;
    pub_twist.publish(msg);
    break;
}
```

디버깅을 하면서 손으로 작성한 구상도.

New 알고리즘.



콜레그는 신호등 2번 인식해야지
범퍼 사용 가능.
(모작중 방지.)

피드백으로 직진 보정.
↳ 직진운동에 문제 있음.

좌회전할 때

현재 좌회전 update.

↳ 그 기준으로 $\pm 90^\circ$

↓
코드 간당.

총 프로젝트 고찰.

이번 학기 프로젝트는 수업 시간에 배운 제어 이론을 ROS 프로그래밍을 통해 로봇한테 적용하는 과정이었다. 첫 번째 프로젝트를 통해 코드로 P 제어기 설계하는 방법에 대한 이해와 기본적인 ROS의 노드, 토픽을 어떻게 사용하는지 정확히 확립했고, 2번째 프로젝트를 통해서 기구학적 제어 모델 설계, 오도 매트기를 이용한 실시간 폐 루프 피드백 시스템을 설계했다. 마지막 프로젝트를 통해 2번 프로젝트 기반 제어 프로그램 설계+영상처리로 실시간 데이터를 받아와 이를 기반으로 판단하고 제어하는 시스템을 구축했다.

특히 2번째 프로젝트를 진행하면서 시뮬레이션에서와 달리 현실에서는 하드웨어(로봇) 간 마찰력, 배터리 상태 등 물리적 오차가 존재하는 걸 확인했다. 이는 시간이 지날수록 오차가 점점 심해져 센서의 정보를 이용한 폐 루프 제어 시스템이 아니면 원하는 동작을 하지 못한다는 걸 몸소 체험할 수 있었다.

임베디드 엔지니어를 지망하는 학생으로서 이번 프로젝트를 통해 단순한 코딩을 넘어 실제로 동작하는 하드웨어에 적용함으로써 실제 로봇 시스템이 갖는 불확실성을 소프트웨어로 제어하고 다양한 센서 정보로 제어를 설계해 오차를 줄이는 방법을 경험함으로써 문제 해결 능력을 키울 수 있어서 정말 소중한 기회였다.